

AI 시대를 방황하는 한 개발자의 연구 일지

본 포트폴리오는 'NC AI'와 '서울경제진흥원'이 주관한 제1회 서울 플레이업 게임 챌린지에 참가하여 3달 간 게임을 만들고 출시 수준으로 폴리싱하는 과정에서, AI를 어떻게 썼는지와 무엇을 배웠는지를 정리한 글입니다.

1

성과 및 결과 요약

게임명	기억결
장르	턴제 헥스 전술 · 덱 빌딩 로그라이크
엔진	Unity 6000.3
기간	2026-05 → 2026-09
팀	3인 (프로그래밍 1, 아트 1, QA 1)
역할	프로그래밍, 기획, UI / UX, 아트 보조
성과	서울 플레이업 AI 게임 챌린지, 전체 353팀 중 상위 8팀 선정 · 2026 GES 전시 및 시상 예정

트레일러 · 인게임 영상 ↗ (<https://www.youtube.com/watch?v=MzOR5wWA2Xk>)

소스코드 및 아키텍처 ↗ (<https://github.com/binnini/memorystone-source>)

2

AI 활용 방식

Unity MCP + Claude

사용한 오픈소스

github.com/IvanMurzak/Unity-MCP

github.com/Yeachan-Heo/oh-my-claudecode

본 프로젝트에서 저는 코드 작성은 물론 씬 생성, 오브젝트 배치, UI 배치까지, 기존 클라이언트 개발자가 하던 작업의 대부분을 오픈소스 Unity MCP로 처리했습니다. `screenshot-game-view`, `screenshot-camera` 같은 스킬로 아트·UI·QA 판정까지 AI가 스스로 수행할 수 있어 빠른 속도로 작업할 수 있었습니다.

Claude는 주로 OMC와 tmux로 **병렬 세션**을 띄워 작업했습니다. 시간이 한정된 공모전 특성상 빠르게 작업하는 것이 중요했기 때문입니다.

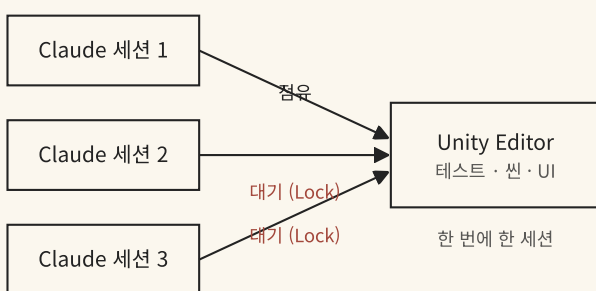
그러나 이 과정에서 '**Unity Editor를 한 번에 한 세션밖에 점유하지 못하는 문제**'가 발생했습니다. 여러 세션이 동시에 에디터를 잡으려 하면 Lock이 걸려 작업이 계속 지연됐습니다. 특히 AI Agent는 테스트 케이스를 많이 작성하는 경향이 있습니다. 테스트는 후속 세션이 이전 작업과 이어지는 다리 역할을 해 주지만, 그 많은 테스트를 병렬 세션이 각자 에디터를 점유해 돌리면 Lock 빈도가 급격히 높아집니다.

이를 해결하기 위해 **게임 규칙은 순수 C#으로만 돌아가고, UI나 오브젝트 배치 같은 표현 계층에서만 에디터를 쓰는 아키텍처**를 택했습니다. 규칙 어셈블리는 `noEngineReferences: true` 로 UnityEngine 참조를 컴파일 단계에서 금지해, 규칙의 구현과 검증이 에디터 없이 dotnet만으로 가능하게 했습니다. 규칙 테스트가 에디터를 점유하지 않으니 테스트 시간도 빨라졌고, 병렬 세션이 에디터에서 충돌할 확률도 줄었습니다.

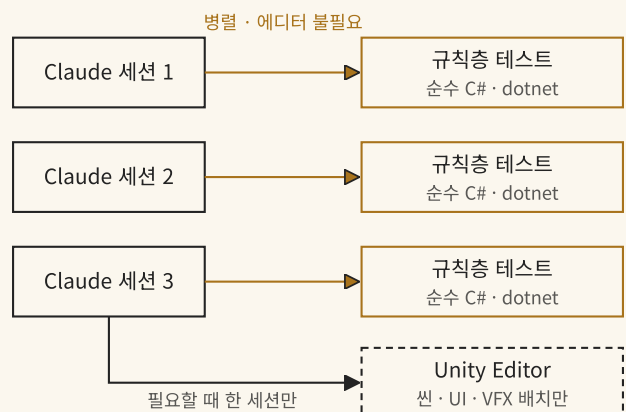
```
asmdef Assets/Scripts/Combat/Runtime/SeoulPlayup.Combat.Runtime.asmdef
```

```
1 {  
2     "name": "SeoulPlayup.Combat.Runtime",  
3     "references": [ "SeoulPlayup.CardCore", "SeoulPlayup.Map.Runtime" ],  
4     "noEngineReferences": true  
5 }
```

전 · 세션마다 에디터를 점유



후 · 규칙은 dotnet에서, 에디터는 표현 계층만



전에는 세 세션이 하나의 Unity Editor를 번갈아 점유해 두 세션은 늘 대기했다. 규칙층을 순수 C#으로 떼어낸 뒤에는 규칙 테스트가 dotnet에서 세션마다 병렬로 돌고, 에디터는 표현 계층을 만질 때만 한 세션이 쓴다.

아트 파이프라인 — 스타일 가이드를 YAML로 제작

공모전 기간 동안 NC AI의 지원으로 'VARCO'를 사용했고, 2D 일러스트는 OpenAI API로 제작했습니다.

UI부터 카드 일러스트, 아이템 아이콘, 3D 모델 원판 이미지까지 모든 일러스트를 AI로 만들었는데, 생성형 AI로 이미지를 만들 때 가장 어려운 것은 **스타일 통일**이었습니다.

저는 프로젝트 안에 스타일 가이드를 YAML로 만들어 두고 레퍼런스가 자동 주입되도록 파이프라인을 구축했습니다. 그 결과 Claude에 프롬프트 한두 줄만 넣고 Claude Web Artifact로 결과를 보며 피드백만 하면 되는 흐름이 만들어졌습니다.

초기에는 결과물이 쌓여 스타일이 고정되기 전까지 퀄리티 편차가 있었지만, 이후 생성물은 별다른 프롬프트 없이도 통일성이 쉽게 확보되었습니다.

style guide tools/imagegen/presets/classes/card_illust.yaml

```
1 preset: card_illust
2 refset: card_illust-v3          # 출하 자산에서 고른 레퍼런스 셋(버전 고정, 불변)
3
4 final: {width: 1448, height: 1086, dest: "Assets/Art/UI/Cards/Illust"}
5
6 stages:
7   explore:
8     model: gpt-image-2
9     endpoint: edits              # 레퍼런스 이미지를 함께 넘기는 경로
10    default_candidates: 4
11
12 art_language:
13   medium: >-
14     flat graphic game illustration in the style of the reference images: bold shapes
15     sealed by a dark navy contour line, high contrast, clean cel shading ...
16   lighting: >-
17     dark cool indigo-violet field with one or two hot saturated light sources
18     inside the frame; strong rim light on the subject; the light source is the subject
19     itself ...
20   motifs: >-
21     Korean folk-shamanic iconography rendered modern: paper talisman slips (부적),
22     fox masks (탈), knotwork (매듭), norigae tassels ... use at most ONE alongside the subject
23   palette:
24     base:  ["#0E1526", "#1B2438", "#080B14", "#2A2160"]
25     accent: ["#2E9BFF", "#7B5BFF", "#FF3B47", "#FF5EC4", "#DCE8FF", "#F0B860"]
26
27 negative:
28   - "card frame, border, title bar, UI frame"
29   - "text, letters, hangul, kanji, watermark"
30   - "photorealistic, photograph, 3d render"
31   - "volumetric fog, atmospheric haze, god rays"
```

request cards-2026-08-07-A14.yaml

```
1 request: cards-2026-08-07-A14
2 class: card_illust
3 targets:
4   - id: A14
5     subject: >-
6       an intimidating threat display - looming shadow, raised weapon and a burst of
7       hot red menace that makes the target cower and weaken
8 variants:
9   - id: shadow
10    refs: ["card_illust_M03.png", "card_illust_A10.png"]
```

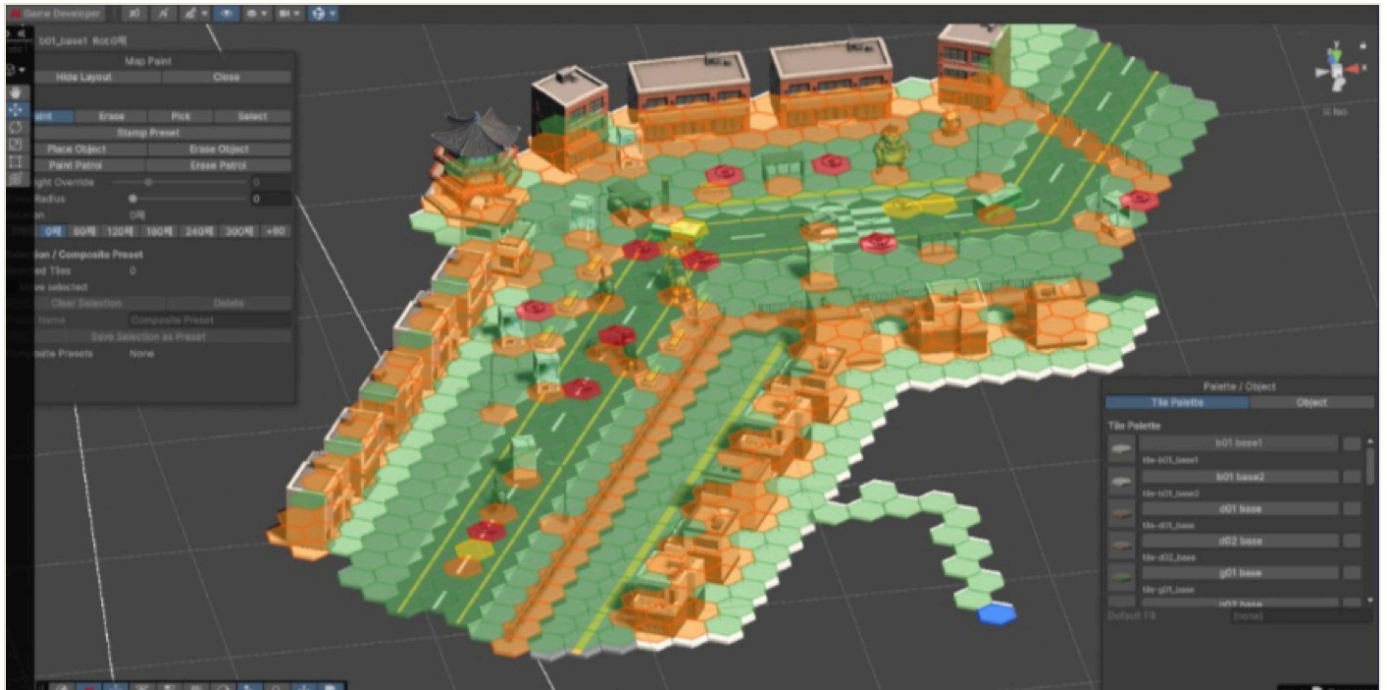


A14 그림장 — 위 요청 파일로 생성된 카드 일러스트. 레퍼런스 두 장(M03 · A10)만으로 스타일 가이드의 인디고 바탕, 탈 모티프, 붉은 광원이 그대로 나왔다.

자체 맵 에디터 등 구현 도구 제작

Unity Editor가 필요한 파트는 AI만으로 완벽한 퀄리티가 나오지 않았습니다. 오브젝트 배치, UI 정렬, VFX 각도, 애니메이션 타이밍처럼 눈으로 확인해야 하는 작업이 그렇습니다. 그래서 사람이 직접 만지고 책임질 수 있는 지점을

만들어 두었습니다. 초기에 런타임 생성 방식이던 UI는 Canvas Prefab으로 바꿔 손으로 조정할 수 있게 했고, 맵 디자인은 Unity MCP로 자체 맵 에디터를 만들어 파일로 관리했습니다. VFX, SFX, 애니메이션도 씬에서 타이밍과 각도를 조절하는 도구를 제작해 설정했습니다.



Unity MCP로 만든 자체 맵 에디터. 타일 팔레트와 오브젝트 배치, 회전, 순찰 경로를 씬에서 직접 칠하고 파일로 저장한다.

3

작업 과정에서 얻은 교훈들

맥락 없는 명령의 대가

세상에 나쁜 AI는 없다. 그러나 나쁜 사람은 있다.

게임 개발에서 초기 기획이 끝까지 그대로 지켜지는 일은 없다고 생각합니다. 기능 추가, 성능 문제, 일정 제약 때문에 처음부터 모든 것을 완벽한 설계 아래 두는 것은 불가능합니다.

AI의 코딩 능력은 굉장히 뛰어나지만, **세션과 메모리에 따라 변동성이 매우 큼니다**. 사람은 여러 세션을 오가며 정제되지 않은 낱것의 요구사항을 던집니다. 하지만 AI Agent 간 세션은 완벽히 공유되지 않습니다. 사람에게는 연관된 두 요구사항이라도, 맥락이 공유되지 않으면 AI는 그 연관성을 파악하지 못합니다. 중간에 기획이 바뀌면, 코드는 처음 설계와 맞지 않는 방향으로 나아갑니다.

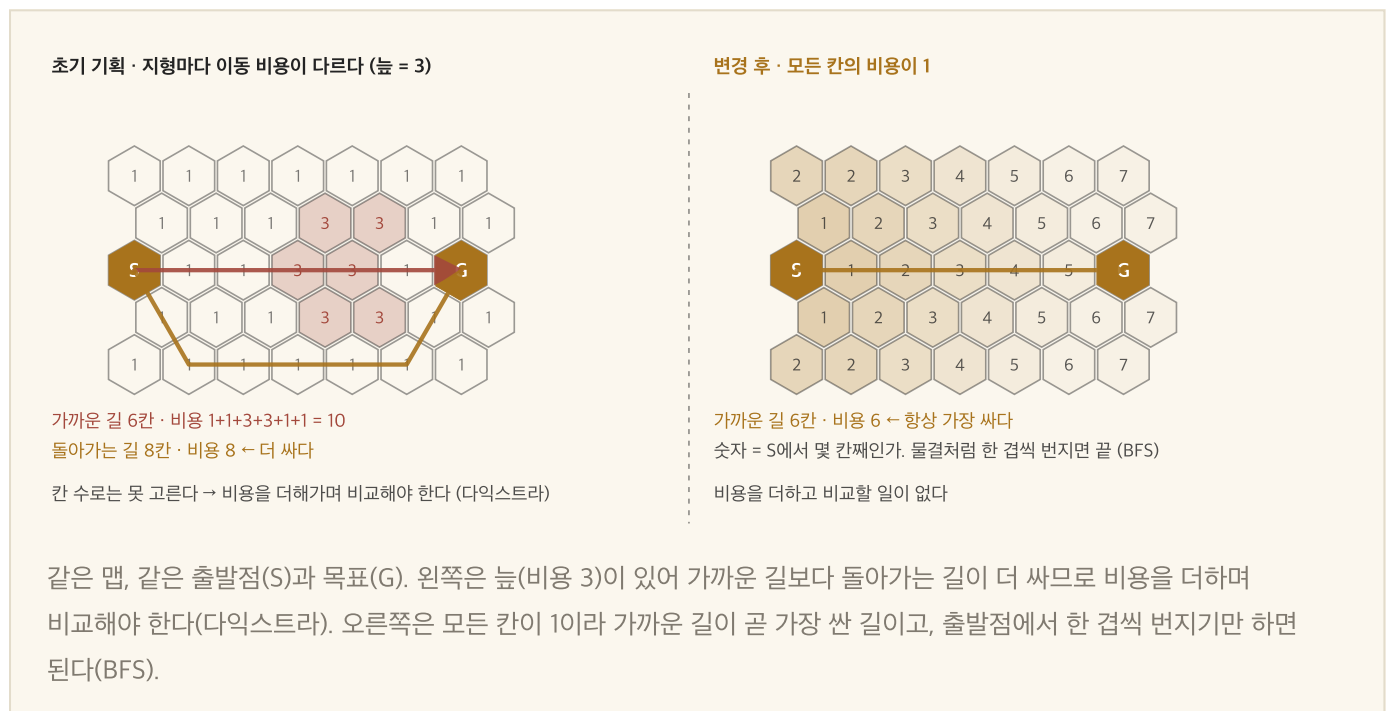
그런데 한 번 AI가 코드의 주도권을 잡으면 사람이 이를 저지하기는 쉽지 않습니다. 실제로 기억결에서는 이 문제로 게임의 메인 규칙을 담당하는 `CombatState` 가 파셜 클래스 38개 파일, 합계 14,860줄에 달하는 God Object가 됐습니다(리뷰 착수 시점 실측).

문제를 인지한 뒤 대대적인 코드 리뷰를 진행했습니다. 기획이 여러 번 수정되며 생긴 수많은 레거시 코드가 있었고, 초기 설계와 아예 다른 방향으로 진행해 버린 케이스도 있었습니다. 세 케이스의 원인은 하나로 수렴합니다. **기획이 중간에 바뀌었는데, AI는 그 변화를 모른 채 이전 전제 위에서 일관되게 계속 작업했습니다.**

대표적인 3가지 케이스를 보겠습니다.

1 기획이 폐지되었음에도 남아 있던 비효율적인 길찾기 로직

초기 기획은 육각 타일의 지형 종류(높 타일 등)에 따라 이동 비용이 달랐습니다. 그래서 경로 탐색에 BFS가 아닌 다익스트라를 썼습니다.



그러나 개발 도중 이 기획이 폐지되어 모든 칸의 이동 비용이 1이 됐습니다. 다익스트라를 쓸 이유가 사라졌지만 AI는 이 변화를 인지하지 못했고, 비효율적인 로직이 그대로 남았습니다. 처음부터 비용이 균일했다면 AI도 당연히 BFS를 골랐을 것입니다. 심지어 남아 있던 구현은 우선순위 큐도 없이, 매 반복마다 미확정 칸 전체를 정렬해 첫 하나만 꺼내는 형태였습니다.

전 · cs:1353 HexPathfinder.CalculateDistances

```
1 var settled = new HashSet<HexCoord>();  
2 distances[query.Start] = 0;  
3  
4 while (true)
```

```

5 {
6     var hasCandidate = false;
7     var current = default(HexCoord);
8     var currentCost = 0;
9     // 미확정 칸 전체를 매 반복 정렬해 놓고 첫 하나만 쓴다
10    foreach (var pair in distances
11        .Where(pair => !settled.Contains(pair.Key))
12        .OrderBy(pair => pair.Value)
13        .ThenBy(pair => discoveryOrder[pair.Key])
14        .ThenBy(pair => pair.Key.Q)
15        .ThenBy(pair => pair.Key.R))
16    {
17        current = pair.Key;
18        currentCost = pair.Value;
19        hasCandidate = true;
20        break;
21    }
22    if (!hasCandidate || currentCost > query.MovePoints) break;
23
24    settled.Add(current);
25    // ... 이웃마다 더 싼 경로인지 비교해 갱신(완화)
26 }

```

후 · cs:1354 HexPathfinder.CalculateDistances

```

1 private const int UniformStepCost = 1;
2
3 var frontier = new Queue<HexCoord>();
4 frontier.Enqueue(query.Start);
5
6 while (frontier.Count > 0)
7 {
8     var current = frontier.Dequeue();
9     var currentCost = distances[current];
10    if (currentCost ≥ query.MovePoints) continue;
11
12    foreach (var neighbor in current.NeighborsInDirectionOrder())
13    {
14        // 균일 비용이므로 첫 발견이 곧 최단이다 - 갱신(relaxation)이 필요 없다.
15        if (distances.ContainsKey(neighbor)) continue;
16        if (!CanEnter(map, current, neighbor, query, runtimeStates, terrainTraits, out var
17            enterCost)) continue;
18        if (enterCost ≠ UniformStepCost) throw new InvalidOperationException("균일 비용 전제가
19            깨졌다");
20
21        distances[neighbor] = currentCost + enterCost;
22        previous[neighbor] = current;
23        frontier.Enqueue(neighbor);
24    }
25 }

```

Stage_1 실제 맵(1,938칸, 도달 가능 875칸)에서 측정한 결과입니다.

케이스	다익스트라	BFS	배수
몬스터 추격 · 최원거리(거리 77)	12.803 ms	0.758 ms	16.9×
도달 불가 · 조기 종료 없음	12.645 ms	0.785 ms	16.1×
플레이어 이동력 4 (19칸)	0.139 ms	0.018 ms	7.7×

이처럼 기획이 중간에 변경되는 경우 AI는 이를 인지하지 못하고 Legacy 코드를 만들어낼 수도 있음을 인지했습니다.

2 시드 기반 재현성 확보의 구현

초기에는 맵과 몬스터 배치가 고정된 스냅샷 방식이었습니다. 기획 중간에 로그라이크 장르를 살리기 위해 시드 기반 배치를 적용하기로 했고, 몬스터 배치와 맵 배치는 시드로 생성되어 UX가 개선되고 재현성을 확보할 수 있었습니다.

하지만 코드 리뷰 결과 AI는 거기서 그쳤고, 전투 판정, 확률 보상, 덱 셔플 같은 세부 영역은 시드 기반으로 바꾸지 않았습니다. 심지어 `System.Random` 과 `UnityEngine.Random` 이 혼용되어 같은 시드로도 다른 판이 나올 수 있었습니다. 배치만 시드로 바꾸라는 요청을 받은 AI는 그 범위만 바꿨고, 「로그라이크니까 전부 재현되어야 한다」는 의도까지는 전달되지 않았습니다.

난수원 전수 조사에서 나온 네 가지 모습입니다. 시드가 없거나, 시드 자리는 있는데 배선이 안 됐거나, 전역 상태를 썼습니다.

```

1 // CardCore/Runtime/DeckState.cs - 덱 셔플. 덱빌딩 로그라이크의 핵심인데 매번 GUID로 시드
2 private readonly Random rng = new Random(Guid.NewGuid().GetHashCode());
3
4 // Combat/Runtime/CombatState.cs - 전투 판정(취약 부위·순간이동지·전염 대상...). 무시드
5 private readonly System.Random pushRng = new System.Random();
6
7 // Combat/Runtime/MonsterAiPlanner.cs - 시드 인자는 있었지만 CombatState가 넘기지 않아 늘 0
8 internal MonsterAiPlanner(IMonsterPlanningContext context, ..., int attackPatternSeed = 0)
9 {
10     monsterAttackPatternRng = new Random(attackPatternSeed);
11     attackDamageJitterRng = new Random(unchecked(attackPatternSeed * 31 + 17));
12 }
13
14 // Combat/Unity/CombatRewardFlow.cs - 보상·상점·뽑기. UnityEngine.Random은 전역 상태라
15 // 연출(카메라 흔들림 등)이 난수를 뽑으면 규칙 결과가 밀린다
16 private sealed class UnityRewardRandom : IRewardRandom
17 {
18     public int Next(int maxExclusive) => Random.Range(0, maxExclusive);
19 }

```



런 시드 하나가 용도별 번호로 갈라져 독립 스트림이 된다. 판정이 몇 번 뽑든 보상과 덱 셔플의 순서는 밀리지 않는다.

이를 해결하기 위해 난수 전체를 시드 기반으로 바꾸고, 런 시드 하나에서 용도(맵, 몬스터, 카드 등)별 번호(0~9)로 스트림 시드를 파생해 쓰도록 했습니다. `RunSeedStreams.Derive(런시드, 스트림번호)` 가 시드를 스트림별로 갈라 냅니다. `Derive(12345, 5)` 는 판정용 시드, `Derive(12345, 8)` 은 이동덱 시드, 이런 식으로 용도별로 나눕니다. 스트림을 나누는 이유는 한 소비자의 뽑기 횟수가 바뀌어도(밸런스 패치, 카드 추가) 다른 소비자의 결과가 밀리지 않게 하기 위해서입니다.

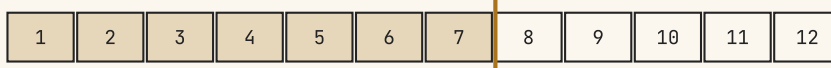
```

1 // 번호는 append-only. 기존 번호를 바꾸면 그 시드로 이미 공유된 판이 달라진다.
2 public static class RunSeedStreams
3 {
4     public const int MonsterPlacement = 0; // 몬스터 슬롯 셔플·풀 추천
5     public const int Traps = 1; // 함정 슬롯 승격·프리셋 치환
6     public const int Chests = 2; // 상자 좌표 셔플
7     public const int Services = 3; // 잡화점·캠핑카 배치
8     public const int SpawnHpVariance = 3; // 스폰 체력 변주(3을 공유하되 spawnRefId로 재혼합)
9     public const int MonsterAttackPattern = 4; // 공격 패턴 선택 + 피해 변주
10    public const int CombatJudgement = 5; // 전투 판정: 취약 부위·순간이동지·전염 대상...
11    public const int BossProps = 6; // 보스 기물 링 회전각
12    public const int Rewards = 7; // 보상·상점·뽑기·전리품
13    public const int MovementDeckShuffle = 8; // 이동덱 셔플 - 행동덱과 갈라 둔다
14    public const int ActionDeckShuffle = 9; // 행동덱 셔플
15
16    public static int Derive(int runSeed, int stream) =>
17        PlacementRandomizer.DeriveSeed(runSeed, stream);
18 }

```

다만 여기서 '세이프 & 로드'가 문제였습니다. 시드 하나를 공유하더라도 카드, 보상, 판정, 몬스터 등 스트림이 여러 개이므로, 세이프 시점에 **각 스트림에서 난수를 몇 번 뽑았는지를 기록하지 않으면** 로드 후 패턴이 세이프 전과 달라집니다.

스트림 5의 뽑기 순서 (CountingRandom)



이미 뽑아 쓴 값 7개

세이프: 커서 = 7

아직 안 뽑은 값

저장하는 것은 시드 12345와 커서 7뿐. 로드: 같은 시드로 새로 만들고 7칸을 버린다

-----> 8번째부터 세이프 전과 똑같이 이어진다

각 스트림은 지금까지 몇 개를 뽑았는지(커서)만 센다. 세이프는 시드와 커서만 담고, 로드는 같은 시드로 새 인스턴스를 만들어 커서만큼 버린 뒤 이어서 뽑는다.

해결을 위해 레퍼런스인 'Slay the Spire'가 이 문제를 어떻게 풀었는지 조사했고 그 방식을 차용했습니다. 난수 내부 상태를 통째로 저장하는 대신 시드와 커서(소비 횟수)만 저장하고, 복원 시 같은 시드로 인스턴스를 새로 만들어 커서까지 빨리감기합니다. 이미 뽑아서 플레이어에게 보인 값(예고된 공격 패턴, 피해 변주)은 체력이나 좌표처럼 값 그대로 저장하고 다시 뽑지 않습니다.

후

Assets/Scripts/CardCore/Runtime/CountingRandom.cs

```

1 // System.Random 파생이라 Random을 받는 기존 호출 자리를 한 줄도 안 고친다.
2 public sealed class CountingRandom : Random
3 {
4     private readonly Random inner;
5     public int Seed { get; }
6     public int Consumed { get; private set; } // 지금까지 소비한 표본 수 = 저장할 커서
7
8     public CountingRandom(int seed) : base(seed) { Seed = seed; inner = new Random(seed); }
9
10    public override int Next() { Consumed++; return inner.Next(); }
11    public override int Next(int max) { Consumed++; return inner.Next(max); }
12
13    public void FastForward(int count) // 복원: 같은 시드로 새로 만들고 여기까지 버린다
14    {
15        for (var i = 0; i < count; i++) inner.Next();
16        Consumed += Math.Max(0, count);
17    }
18 }

```

후

Assets/Scripts/Combat/Runtime/CombatState.RngCursors.cs

```

1 // 로드: 스트림마다 같은 시드로 새로 만들고, 저장된 커서만큼 버린다
2 private void RestoreRngCursors(RngCursorsSaveData cursors)
3 {
4     if (!RunSeed.HasValue) return;
5
6     pushRng = CreateStreamRandom(RunSeedStreams.CombatJudgement);
7     ((CountingRandom)pushRng).FastForward(cursors.Judgement);
8     planner.RestoreRngCursors(cursors.AttackPattern, cursors.DamageJitter);
9     bossPropRng = CreateStreamRandom(RunSeedStreams.BossProps);
10    bossPropRng.FastForward(cursors.BossProps);
11    movementShuffleRng = CreateStreamRandom(RunSeedStreams.MovementDeckShuffle);
12    movementShuffleRng.FastForward(cursors.MovementShuffle);
13    actionShuffleRng = CreateStreamRandom(RunSeedStreams.ActionDeckShuffle);
14    actionShuffleRng.FastForward(cursors.ActionShuffle);
15 }

```

3

CSV 기반 카드 구현에서 클래스 기반 카드 구현으로

카드는 처음에는 CSV 기반 데이터 주도로 설계했습니다. `Cards.csv` 한 행에 카드의 규칙을 기술하고, 코드는 `behaviorId` 문자열로 별도 핸들러를 찾아 실행하는 구조였습니다. 코드를 모르는 팀원이 CSV를 통해 카드를 만들 수 있게 하려는 의도였습니다.

그러나 실제로는 카드 저작을 비개발자와 나눌 일이 없었고, 카드 고유 효과는 점점 복잡해졌습니다. 그런데 AI는 CSV 기반으로 계속 작업했습니다. 새 카드에 고유 규칙이 필요할 때마다 코드 대신 CSV 컬럼에 문자열 토큰을 밀어 넣었고, `postActions=SpreadStatus:2`, `additionalCost`, `choiceOptions`, `behaviorParams` 같은 컬럼이 계속해서 늘어나 파서가 458줄이 됐고, 규칙 하나를 추가하려면 핸들러 클래스, 허용 목록, CSV, VFX 큐, 키워드 문안 다섯 곳을 건드려야 했습니다.

1가지 예를 들자면, 전염병(A12) 카드가 이전에는 CSV 행, 파서, 핸들러 맵, 핸들러 클래스, `CombatState` 분기 다섯 곳으로 분산되어 있었습니다. AI가 CSV 기반으로 효과를 확장하려다 보니 구조가 너무 복잡해진 것이 원인이었습니다.

전 · cs:1384 A12 전염병 - CSV 행 · 핸들러 클래스 · CombatState 분기

```
1 // cards.csv (32열) - 규칙은 behaviorId와 postActions 토큰이 든다
2 // id,name,...,behaviorId,...,postActions,...
3 // A12,전염병,...,attack.plague,...,SpreadStatus:2,...
4
5 // CombatState.AttackCardEffects.cs - behaviorId 문자열로 찾아지는 핸들러. 피해 계산만 여기
6 private sealed class PlagueAttackEffectHandler : IAttackEffectHandler
7 {
8     public string EffectRef => CardEffectRefs.AttackPlague;
9
10    public int GetDamage(CombatState state, CardDefinition card, int attackBonus, HexCoord
    target)
11    {
12        var monster = state.FindLivingMonsterAt(target);
13        var afflicted = monster != null && state.HasCleansableDebuff(monster.Id);
14        return Math.Max(0, card.Amount + (afflicted ? card.Amount : 0) + attackBonus);
15    }
16 }
17
18 // CombatState.cs - 전염 반경은 CSV 토큰을 다시 파싱해서 결정
19 if (string.Equals(action.ActionId, CardBehaviorMetadata.PostActionSpreadStatus,
    StringComparison.Ordinal))
20 {
21     SpreadTargetDebuffsToNeighbours(card, target, int.TryParse(action.Payload, out var
    radius) && radius > 0 ? radius : 1);
22     continue;
23 }
```

코드 리뷰 후, 고유한 카드 효과를 더 쉽게 만들기 위해 카드당 C# 클래스 하나 형태로 변경했습니다. CSV에는 이름, 문안, 비용, 피해량 같은 표시와 밸런스 값만 남기고, 카드의 규칙은 클래스에서 모두 수행하도록 했습니다.

```

1 // cards.csv (26열) - behaviorId.postActions 등 규칙 컬럼 다섯 개가 없다
2 // A12, 전염병, ..., living_monster_in_range, Approved, TRUE, TRUE, card_illust_A12, Legendary,,
3
4 public sealed class A12_Plague : BasicAttackCard
5 {
6     public override string Id => "A12";
7     public override IReadOnlyList<string> Keywords { get; } = new[] { "전염" };
8     public override CardDefinition Upgrade(CardDefinition card, int level) =>
9     card.With(amount: 5);
10
11     public const int SpreadRadius = 2; // 옛 postActions=SpreadStatus:2
12
13     public override int GetAttackDamage(CombatState state, CardDefinition card, int
14     attackBonus, HexCoord target)
15     {
16         var monster = state.FindLivingMonsterAt(target);
17         var afflicted = monster != null && state.HasCleansableDebuff(monster.Id);
18         return Math.Max(0, card.Amount + (afflicted ? card.Amount : 0) + attackBonus);
19     }
20 }

```

결론

3달간 게임을 개발해오면서 AI를 많이 사용하고 연구해본 결과, AI 하나로 정말 많은 것들을 할 수 있다는 것을 다시 느꼈습니다. AI가 없었더라면 저는 현재 수준의 10%도 만들지 못했을 것이라고 확신합니다.

하지만 AI가 완벽하지 않다는 것도 느꼈습니다. AI가 사람의 생각을 완벽히 이해할 것이라는 안일한 생각은 버려야 하며, AI가 제시한 아키텍처나 해결 방안이 때로는 잘못됐을 수 있다는 의심을 항상 가져야 한다는 것을 배웠습니다.

AI Native 시대의 개발자들은 코드를 쓰는 것이 아니라, 요구사항의 변경이 있을 때 그것이 전체 아키텍처에 어떤 영향을 미칠지 알아채고 AI에게 알려줄 수 있어야 하고, 주기적인 코드 리뷰 등 그것이 가능한 환경을 미리 만들어 놓는 것이 중요하다는 것을 배웠습니다.